Lab 11 — Waarom Kubernetes: van Compose naar het cluster

Theorie — wat Compose nooit zal kunnen, wat een cluster is, waarom je er een toestand beschrijft in plaats van commando's te geven, en hoe k3s dat alles in één minuut in je WSL zet.

Doelstellingen

- De beperkingen van Compose benoemen die een orchestrator rechtvaardigen.
- Een cluster beschrijven: *control plane* en nodes, en de rol van elk onderdeel.
- Het **declaratieve model** en de **reconciliatielus** begrijpen.
- In de Kubernetes-Pod de Podman-pod van lab 07 herkennen.
- k3s installeren op WSL, met `kubectl` praten, en een Podman-image in het cluster krijgen.

---

1. Wat Compose niet kan

In lab 09 zette `podman compose up -d` de hele stack op je machine. In lab 10 zag je wat er voor productie ontbrak: valt de machine uit, dan valt alles uit; heeft de API drie instanties nodig op drie machines, dan weet Compose van niets; moet een versie de vorige vervangen zonder de dienst te onderbreken, dan script je het zelf; en sterft een container, dan herstart `restart` hem — maar op dezelfde plaats, met dezelfde panne. Die vier tekorten hebben een naam: **hoge beschikbaarheid**, **spreiding**, **geleidelijke update**, **zelfherstel**. Een orchestrator bestaat om ze te leveren.

> **→ GESCHIEDENIS**
> Google draait zijn diensten sinds 2003 in containers met een intern systeem, **Borg**, dat miljoenen taken verdeelt over honderdduizenden machines. In 2014 publiceren drie Google-ingenieurs (Joe Beda, Brendan Burns, Craig McLuckie) er een open herschrijving van: **Kubernetes** (Grieks voor *stuurman*, afgekort "k8s"). In 2015 geschonken aan de Cloud Native Computing Foundation, wordt het in vijf jaar de feitelijke standaard onder de orchestrators, vóór Swarm en Mesos. **k3s** (Rancher, 2019) is een gecertificeerde, uitgeklede distributie voor de edge, IoT en ontwikkelmachines.

2. Een cluster: één brein en vele armen

Een **cluster** is een verzameling machines (*nodes*) die containers uitvoeren, aangestuurd door een **control plane** dat beslist waar en hoe.

Onderdeel	Rol	De Podman-analogie
API server	de enige toegangsdeur: ontvangt, valideert en bewaart elk verzoek	de CLI `podman`, maar als HTTPS-server
etcd	sleutel-waardedatabase met de gewenste en de waargenomen toestand	de lokale database van Podman
scheduler	kiest op welke node elke Pod komt	niemand: je eigen machine
controller manager	de lussen die gewenst en waargenomen vergelijken en bijsturen	`--restart`, maar veralgemeend naar alles
kubelet (op elke node)	de agent die de gevraagde Pods laat draaien	`podman` + `conmon`
container runtime (containerd, via CRI)	maakt de containers echt aan	`crun`
kube-proxy (op elke node)	het netwerk van de Services (lab 13)	`rootlessport`

Alle communicatie loopt via de **API server**: `kubectl`, dashboards, controllers, de kubelet zelf. Dat maakt Kubernetes uitbreidbaar — alles wat HTTPS spreekt, kan het aansturen — en daarom is de eerste vraag bij een diagnose "antwoordt de API server?" (`kubectl cluster-info`).

> **K3S**
> k3s stopt **het hele control plane in één binary** van 70 MB, vervangt etcd door SQLite (voldoende voor één node), bouwt containerd in, en voegt toe wat een kaal cluster mist: een Ingress controller (Traefik), een load balancer (ServiceLB), een opslagklasse (`local-path`), CoreDNS en metrics-server. Het draait als **root**, als een systemd-dienst — het is de kubelet die de containers aanmaakt, niet jij. Een bedrijfscluster verdeelt die onderdelen over meerdere machines; de commando's en de manifesten zijn **dezelfde**.

3. Het declaratieve model

Met Podman geef je **bevelen**: `run`, `stop`, `rm`. Met Kubernetes leg je een **beschrijving** neer: "ik wil een Pod met de naam `api`, met deze image, deze poort, deze variabele". De API server bewaart ze; de controllers en de kubelet werken tot de werkelijkheid ermee overeenstemt; en ze **blijven** toezien. Verdwijnt de Pod, dan wordt het verschil tussen gewenst en waargenomen opgemerkt en gecorrigeerd. Dat is de **reconciliatielus**, het hart van heel Kubernetes.

Een object beschrijf je in YAML met vier sleutels: `apiVersion` (de groep en de versie van de API), `kind` (het type: Pod, Deployment, Service…), `metadata` (naam, namespace, labels), `spec` (wat je wilt). Het cluster voegt er `status` aan toe (wat het waarneemt). `kubectl apply -f bestand.yaml` stuurt de spec; `kubectl get pod api -o yaml` geeft spec **en** status terug.

> **ONTHOUDEN**
> `kubectl apply` is **idempotent**: tien keer uitgevoerd verandert het niets als het bestand niet veranderd is, en het werkt bij als het bestand wel veranderd is. `kubectl create` maakt één keer aan en faalt de tweede keer (`AlreadyExists`). Een geversioneerd manifest + `apply` = dezelfde discipline als Compose, maar toegepast door een systeem dat nooit stopt met controleren.

In lab 11 is de losse Pod nog een object **zonder controller**: verwijderd, komt hij niet terug. Het is het Deployment (lab 12) dat de lus "ik wil N replica's" draagt. Je ziet het aan het einde van het

praktijklab: een Pod die je onder een Deployment wist, verschijnt binnen enkele seconden opnieuw.

4. De Pod

De eenheid van uitrol is niet de container maar de **Pod**: een of meer containers die een IP-adres, een `localhost` en volumes delen. Je maakte er al een in lab 07 met `podman pod create`: hetzelfde begrip, hetzelfde woord, hetzelfde gedeelde `/etc/hosts`. Een Pod krijgt een IP uit het clusternetwerk (`10.42.0.x` bij k3s); dat verandert bij elke nieuwe aanmaak, en geen enkele client mag het vast inschrijven — dat wordt de rol van de Services (lab 13).

> **→ DOCKER-HERHALING**
> Een Podman-container = een geïsoleerd proces; een Podman-pod = meerdere containers in dezelfde network namespace; `podman generate kube` schrijft die pod als Kubernetes-YAML en `podman kube play` leest hem terug. De Kubernetes-Pod is precies dat object, door een scheduler op een node geplaatst en door een kubelet bewaakt. Wat je over containers wist, blijft binnenin waar: PID 1, signalen, exitcodes, logs op stdout, omgevingsvariabelen.

> **→ NETWERK**
> Elke node krijgt een adresbereik voor zijn Pods (`10.42.0.0/24` op de eerste k3s-node), en een **CNI** (*Container Network Interface*, hier flannel) verbindt de bereiken van de nodes met elkaar: elke Pod bereikt elke Pod op zijn IP, zonder NAT. Het is het user-defined netwerk van lab 07, uitgebreid naar meerdere machines. De host houdt zijn eigen adres (`hostIP` in de status van de Pod).

In de praktijk hebben de meeste Pods **maar één** container; de tweede is een *sidecar* (logagent, TLS-proxy) of een *init container* (schemamigratie vóór de API).

5. `kubectl` en de kubeconfig

`kubectl` leest een **kubeconfig**-bestand (`~/.kube/config`, of `KUBECONFIG`, of voor k3s `/etc/rancher/k3s/k3s.yaml`) met het adres van de API server, het certificaat van de autoriteit en je inloggegevens; een **context** koppelt een cluster, een gebruiker en een standaard-**namespace**. Namespaces schermen objecten af; in deze reeks werkt elk lab in het zijne (`kubectl create namespace lab11`, dan `kubectl config set-context --current --namespace=lab11`).

Commando	Gebruik
`kubectl get pods` (`-o wide`, `-o yaml`, `-A`)	oplijsten, met node en IP, of het volledige object, of alle namespaces
`kubectl describe pod api`	details + **Events**: de chronologie van wat het cluster gedaan heeft
`kubectl logs api`, `kubectl exec api -- cmd`	zoals `podman logs` / `podman exec`
`kubectl apply -f`, `kubectl delete -f`	manifesten neerleggen / weghalen
`kubectl run web --image=…`	een Pod maken zonder bestand (snelle test)
`kubectl port-forward pod/api 18080:8080`	lokale tunnel naar een Pod, zonder poort te publiceren
`kubectl explain pod.spec.containers`	de documentatie van elk veld, offline
`kubectl api-resources`	alle objecttypes van het cluster

→ WINDOWS / WSL
Installatie in één regel, in je Ubuntu: `curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644`. Het script installeert de binary, een systemd-dienst `k3s` (dus `systemd=true` is verplicht) en een `kubectl`. `--write-kubeconfig-mode 644` maakt de kubeconfig leesbaar zonder `sudo`: handig op een werkstation, onaanvaardbaar op een gedeelde server (dat bestand **is** de beheerderstoegang tot het cluster). Reken op 4 GB RAM voor WSL (`.wslconfig`, `memory=6GB` aangeraden); `wsl --shutdown` stopt het cluster, systemd herstart het bij de volgende `wsl`.

6. Een image in het cluster krijgen

Het cluster heeft **zijn eigen** imageopslag, die van de containerd van k3s (als root, in `/var/lib/rancher/k3s`). Podman heeft de zijne (`~/.local/share/containers`). Een `podman build` geeft het cluster dus niets: `kubectl apply` van een Pod op `localhost/api:1.0` eindigt in `ErrImagePull` en dan `ImagePullBackOff`, want de kubelet probeert de image te **halen** uit een registry met de naam `localhost`. Twee oplossingen: ze importeren (`podman save localhost/api:1.0 | sudo k3s ctr images import -`), wat deze reeks doet; of ze pushen naar een registry die het cluster kent (lab 17). In beide gevallen zegt `imagePullPolicy: IfNotPresent` de kubelet om de lokale image te gebruiken als ze bestaat; `Always` (de standaard voor de tag `latest`) zou bij elke start een pull afdwingen.

PODMAN
Podman spreekt Kubernetes van nature: `podman generate kube` schrijft je pods als YAML, `podman kube play` leest een Kubernetes-manifest en maakt de overeenkomstige pods aan zonder cluster. Het geproduceerde bestand bevat wel dingen die aan één machine gebonden zijn (`hostPath`, `hostPort`) en die een cluster vervangt door ConfigMaps en Services. En `sudo k3s crictl ps` / `sudo k3s crictl images` tonen de containers en images **van het cluster**, waar `podman ps` alleen de jouwe toont.

7. In het bedrijf

- **De ontwikkelaar schrijft manifesten, het platformteam laat het cluster draaien.** Je krijgt een kubeconfig met een context per omgeving en een namespace per team; het control plane zie je nooit.
- **Een beheerd cluster** (EKS, AKS, GKE) of OpenShift vervangt k3s in productie: control plane beheerd door de leverancier, nodes op aanvraag, load balancers en opslag aangesloten op de cloud. Je manifesten veranderen niet.
- **Compose blijft op het werkstation**; Kubernetes begint bij de eerste gedeelde omgeving. Veel teams houden beide, en een `kubectl apply` in CI doet wat `compose up` op het werkstation doet (lab 17, GitOps).
- **De woordenschat telt**: Pod, Deployment, Service, Ingress, namespace, manifest, reconciliatie. Het is die van incidenten, reviews en sollicitatiegesprekken.

Onthouden

- Compose doet geen hoge beschikbaarheid, geen spreiding, geen geleidelijke update en geen zelfherstel: dat is het werk van een orchestrator.
- Een cluster = een control plane (API server, etcd, scheduler, controllers) + nodes (kubelet, containerd, kube-proxy); alles loopt via de API server.

- Kubernetes is **declaratief**: je legt een gewenste toestand neer, reconciliatielussen houden ze in stand; `apply` is idempotent.
- De Pod is de eenheid van uitrol — de Podman-pod van lab 07 — met een vluchtig IP uit het clusternetwerk.
- `kubectl` leest een kubeconfig, werkt in een context en een namespace; `describe` en zijn Events zijn het eerste diagnosemiddel.
- Het cluster heeft zijn eigen imageopslag: `podman save | sudo k3s ctr images import -`, en `imagePullPolicy: IfNotPresent`.
- k3s = heel Kubernetes in één binary, met Traefik, ServiceLB en `local-path`; de manifesten zijn die van elk cluster.

Woordenschat

orchestrator: systeem dat containers op meerdere machines plaatst, bewaakt en herstelt. — **cluster / node**: verzameling machines / één machine die Pods uitvoert. — **control plane**: API server, etcd, scheduler, controller manager. — **kubelet**: node-agent die de Pods laat draaien. — **CRI / containerd**: interface en containerruntime van de node. — **manifest**: YAML-bestand dat een object beschrijft (`apiVersion`, `kind`, `metadata`, `spec`). — **gewenste / waargenomen toestand**: `spec` / `status`. — **reconciliatielus**: controller die het verschil tussen beide corrigeert. — **Pod**: groep containers die IP, `localhost` en volumes delen. — **CNI**: netwerkplugin die de Pods over de nodes heen verbindt. — **kubeconfig / context / namespace**: toegang tot het cluster / cluster + gebruiker + standaardnamespace / afscherming van objecten. — **`imagePullPolicy`**: wanneer de kubelet een image haalt. — **k3s**: gecertificeerde Kubernetes-distributie in één binary.